GOSIM

GOSIM Shenzhen 2026

Security Agents

An Autonomous Runtime Shielding Agent for Linux Kernel Vulnerabilities

Dongbiao He | Southeast University

GOSIM Technical Talk



CONTENTS

Part 01 Why We Need a Runtime Shielding Agent

Part 02 What Is the Runtime Shielding Agent

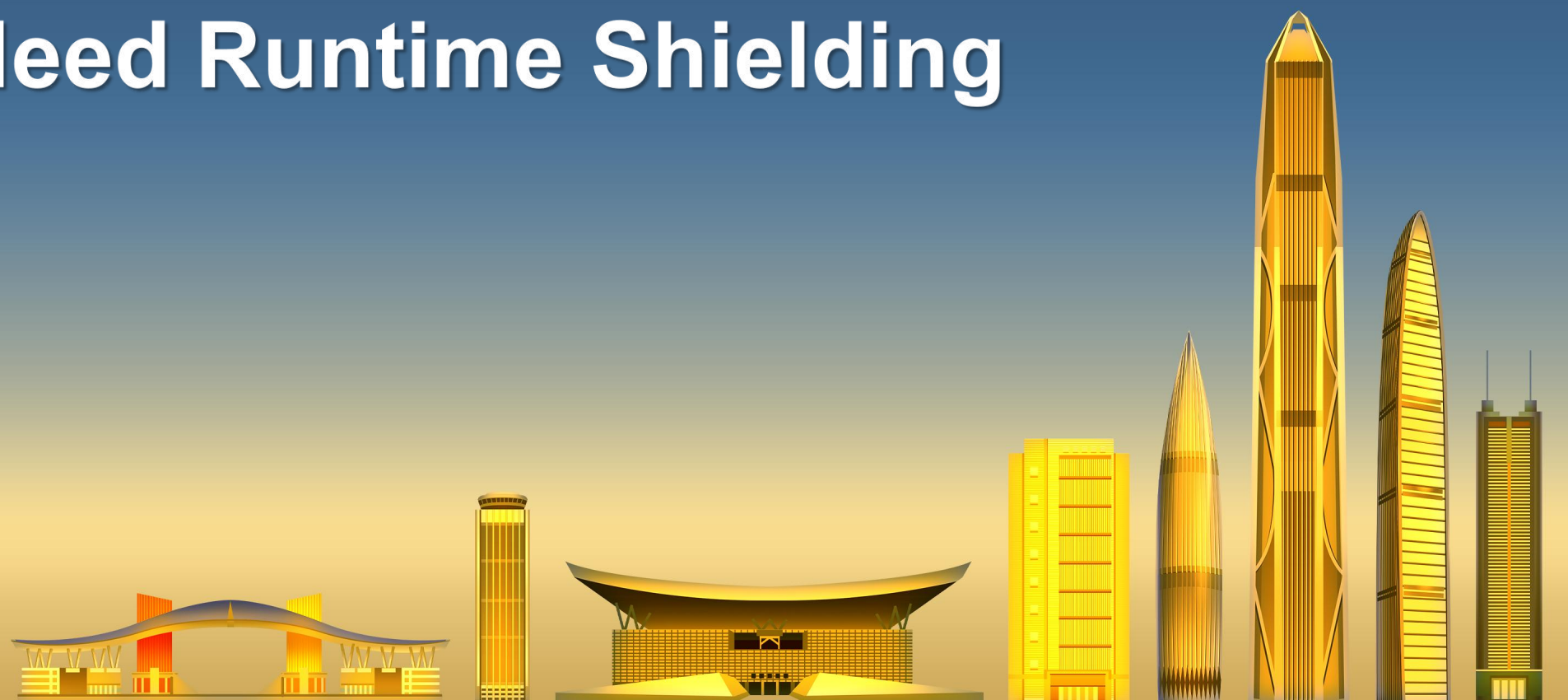
Part 03 How the Agent Proves Itself

Part 04 Does It Hold Up, and What's Next

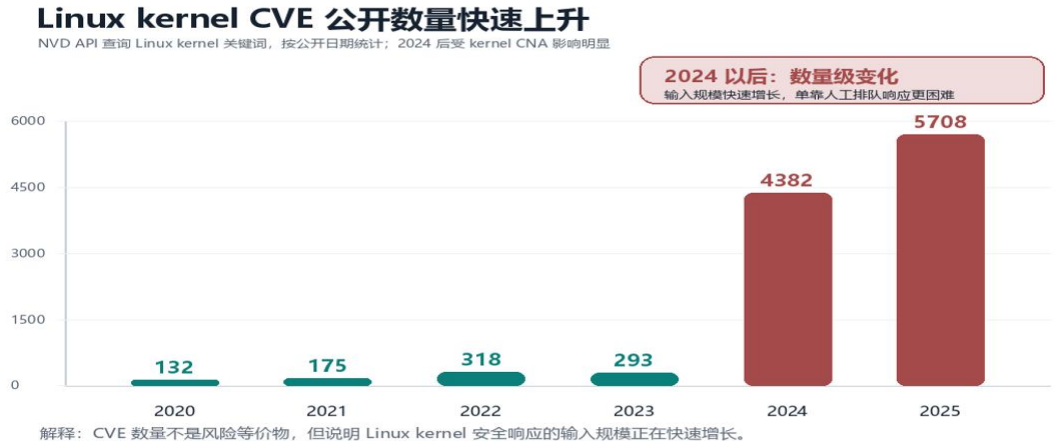


01

Why We Need Runtime Shielding Agent



AI Is Compressing the Vulnerability Response Window





140 Windows CVEs
Microsoft FORGE helped discover Windows vulnerabilities assigned 140 CVEs from May to September 2026.

155 validated OSS reports
FORGE submitted 155 internally validated reports across 23 open-source projects, including the Linux kernel..

5,045 → 10,740 disclosures/month
Google GTIG reported monthly vulnerability disclosures doubled from January to August 2026.

10.5 → 18 exploited/month
GTIG reported exploited vulnerabilities increased from 10.5/month in 2025 to 18/month in January-August 2026. (cloud.google.com)

Why Agents Need Expertise

Between public disclosure and a tested, rebooted rollout, the kernel runs a known-exploitable bug in production.



Disclosure → rollout is days to weeks

A tested reboot is slow

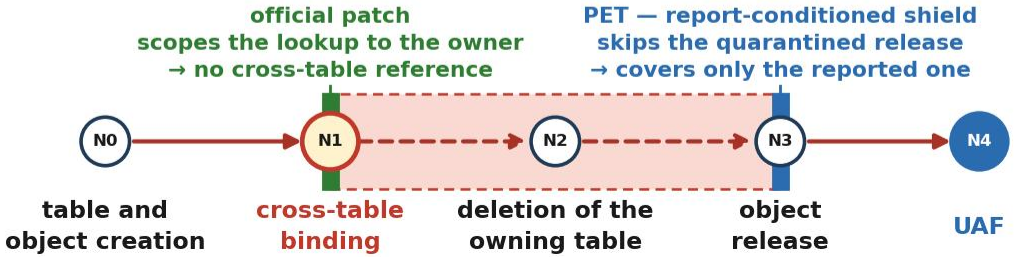
You need a hot shield now

The Problem(1)

```
@@ -3842,6 +3842,7 @@
 static struct nft_set *
 nft_set_lookup_byid(const struct net *net,
+                    const struct nft_table *table,
                     const struct nlattr *nla, u8 genmask)
@@ -3853,6 +3854,7 @@
         struct nft_set *set = nft_trans_set(trans);
         if (id == nft_trans_set_id(trans) &&
+            set->table == table &&  (1)
             nft_active_genmask(set, genmask))
@@ -3873,7 +3875,7 @@
         if (!nla_set_id)
-        set = nft_set_lookup_byid(net, nla_set_id, genmask);
+        set = nft_set_lookup_byid(net, table, nla_set_id, genmask);
```

(1) restores the ROOT CAUSE — a set owned by one table cannot cross to another.

(a) Official patch (held out) — enforces the root-cause invariant




```
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 40960);  (2)
    __type(key, u64);
    __type(value, u64);
} quaran_obj_map SEC(".maps");
...
SEC("kprobe/kfree")
int BPF_KPROBE(initialized)
{
    pval = bpf_map_lookup_elem(&quaran_flag, &pid);
    if (pval != NULL) {
        u64 obj_addr = (u64) ctx->di;
        ...
        err = bpf_map_update_elem(&quaran_obj_map,
                                  &obj_addr, &v, BPF_ANY);
        if (err < 0)
            return -1;
        bpf_override_return(ctx, 0);  (3)
    }
}
```

(2) Bypass 1 — the quarantine map is fixed-size: once it is full, later releases are neither recorded nor stopped, so the cut is skipped.

(3) Bypass 2 — the cut lands at the symptom layer: the cross-table reference is already made, so any other release path escapes it.

Blocking the crash is not holding the cause

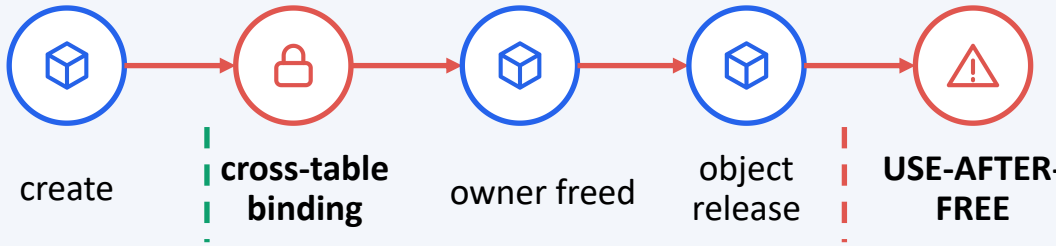
Crash site ≠ root cause

Swap the trigger, reach the same bug

LLMs make variant search cheap

Attackers enumerate bypasses at low cost.

ONE ROOT CAUSE · MANY TRIGGERS

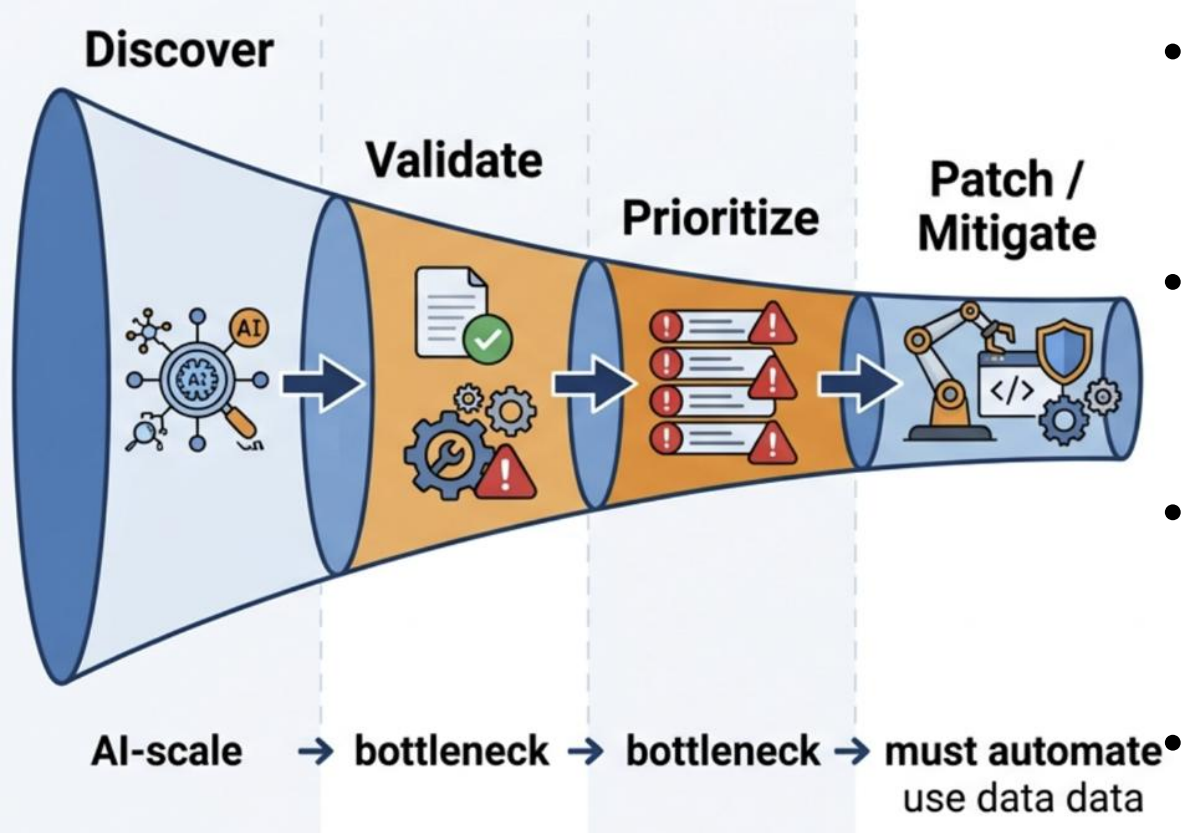


Root-cause cut
holds every variant

Symptom cut
one path escapes it

Motivating case · CVE-2022-2586 (nf_tables UAF)

The Requirement



- **Discover**：AI-scale, "machines flag thousands of candidates"
- **Validate**：human bottleneck, "reproduce, confirm, find root cause"
- **Prioritize**：human bottleneck , "triage by impact & exploitability"
- **Patch/Mitigate**：must automate ,"synthesize fixes / runtime shields at scale"

02

What Is the Runtime Shielding Agent



Give the whole loop to an agent

CairShield is two things at once: a system-level defense runtime, and the open artifact it produces.

AGENTIC OS

An autonomous defense runtime

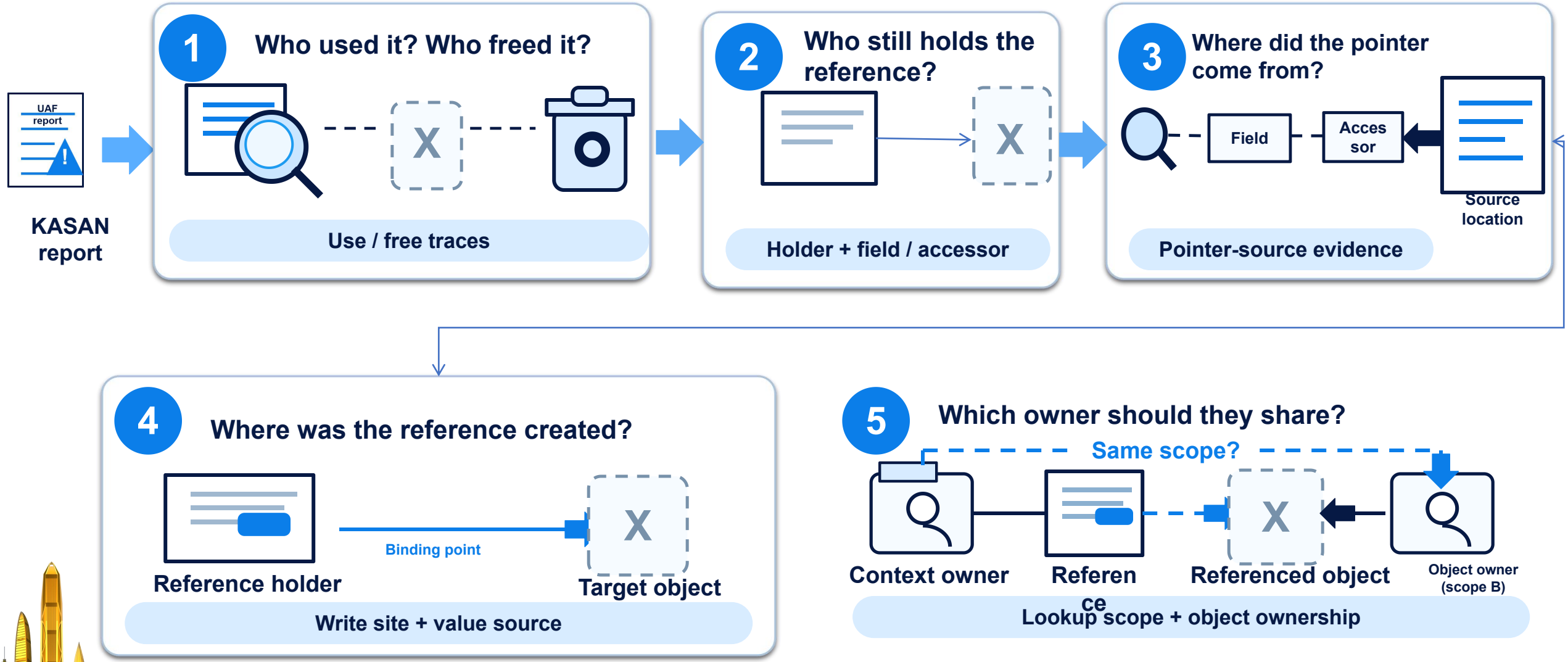
Perceive	reproduce the crash, gather runtime evidence
Reason	infer the violated invariant, not the symptom
Act	synthesize & hot-load an eBPF shield, zero downtime
Evolve	attack its own shield, refine until it holds

AGENTIC APP

A verifiable, shippable artifact

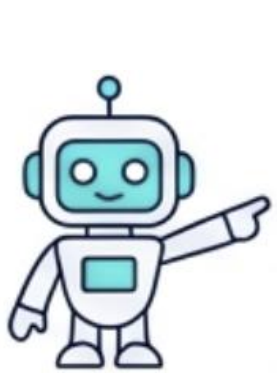
Validated	only shields that survive self-test are released
Hot-deployable	attach to a live kernel, roll back instantly
Open	every shield auditable & reproducible in a repo
Low overhead	≤1.39% on general workloads

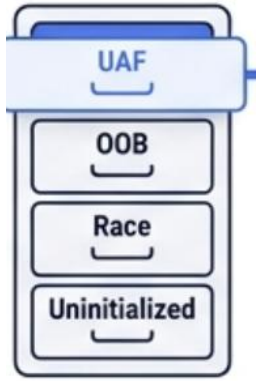
Turn Knowledge into a Reasoning Map



How the Agent Uses the Knowledge

1. Choose the knowledge branch

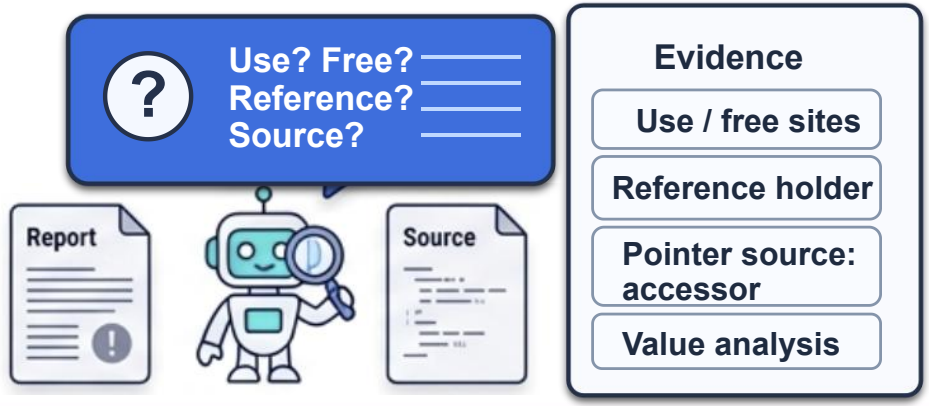




Cheklist

- Call stack analysis
- Object lifetime review
- Data flow tracing

2. Investigate with guidance



3. Select the subtype knowledge

UAF subtype knowledge

- Container/registry → Missing unlink
- Named slot → Missing nullout
- Accessor → Cross-owner

Root Cause Evidence

Anlysis Chain

Reasoning Strategy

4. Interpret evidence with expertise

Expert strategy

Case evidence



Defense semanti cs

Reference holder (Scope A)

Object (Scope B)

What Knowledge Decides & What the Agent Infers

① What Knowledge Decides

Question chain — what to ask, in order

find where the reference was created · what keeps the object alive

Which evidence source answers each question

use point · free point · binding point

The invariant SHAPE (by construction)

`referenced.scope == context.scope`

Repair-obligation form

unchecked → deny · mischecked → override

Written once per subtype — reused across every vulnerability of that shape.

guidance + grounded facts

② What the Agent Must Infer (one bounded step)

template token — abstract shape — steps (1)–(4) are deterministic

`referenced.scope == context.scope`

instantiate with case fields — step (5), the model's only task

grounded-facts

- owner: table A
- holder: table B
- release: A teardown

`set->table == table`

scope container = nft_table · repair = deny

may NOT invent a new invariant kind

cannot certify its own output — a deterministic gate re-checks every operand

Cross-owner reference strategy

Use ownership and reference information to find binding sites and trace object lifetime.

1. Evidence supports the rule

Binding located	Object Freed by A	Reference Survives in B



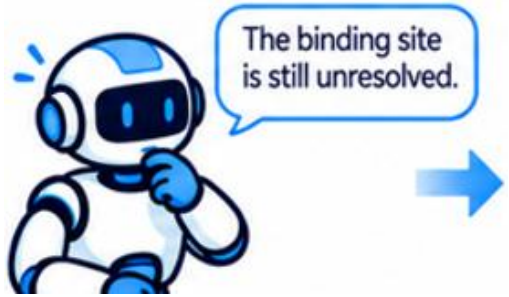
Proceed With a grounded candidate

2. Key evidence is missing

Surviving Reference

Write Site A?

Write Site B?



Keep the uncertanty explict

3. Evidence Challenges the rule

Shared liftetime guarantee

Owner A

Owner B

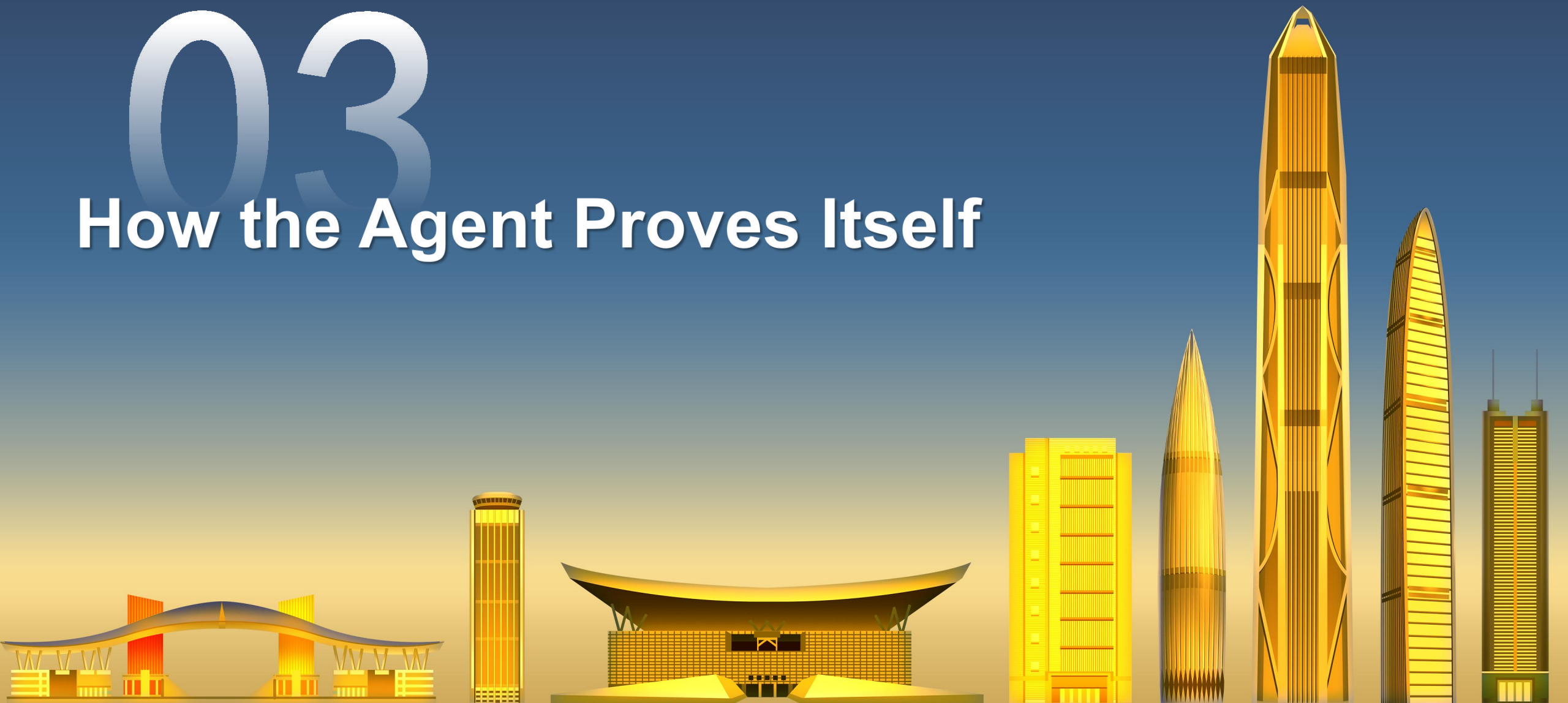


Do not apply the rule blindly

GOSIM

03

How the Agent Proves Itself



Probe the boundary, not the trigger

The agent attacks what it just built : keep the root cause violated, step outside the shield.

MONOSPACE INVARIANT current shield = symptom-layer

```
set->table == table
```

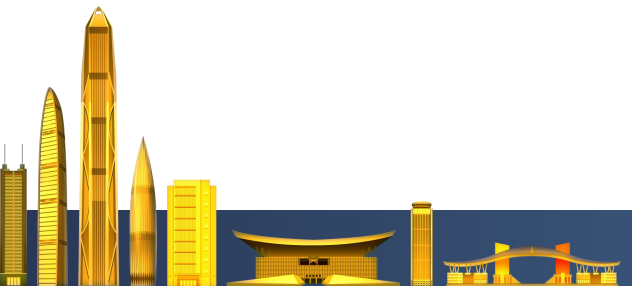
must stay violated · CVE-2022-2586

THE MOVE

- overflow the fixed-size quarantine → later free never recorded
- free via a path the shield never watches

4 built · 4 escaped

- A-PoC → MUST BLOCK (C = k/N)
- B-PoC → MUST ALLOW (P = 1 − m/M)



A bypass is a diagnosis, not a verdict

The agent localizes the gap a counterexample exposed :
then binds itself to a repair obligation.

THE BYPASS

raw failure trace

```
[2024-05-15T10:21:33.456Z] TRACE ....>
...
`validate_transaction(txn: Transaction)
    assert(txn.recipient == allowed_recipient)
    assert(txn.amount <= limit)
    `set->table == table` (VIOLATED)
...
...> ALLOWED
```

allowed — root cause preserved

LOCALIZE

- Expressibility **(C1)**: the predicate can't name this release path
- Localizability **(C2)**: operand not grounded to an observed event

OBLIGATION

repair obligation → deny / override

scoped note

close THIS coverage gap

- keep everything that already held

feedback is a hypothesis bounded by evidence — not a free rewrite

Revise the shield, replay the whole suite

The agent edits the predicate and re-grounds operands — acceptance is earned, never asserted.

REVISE

```
invariant_shield(
    target = 'nft_set:  set->table == table',
-   bind_tbl = 'table @ transaction.commit'
+   bind_tbl = 'table @ nft_set_lookup_byid:entry'
)
```

REPLAY

1. load shield
2. reported trigger + A-PoCs
3. unload
4. B-PoCs + controls
5. baseline last
6. revert snapshot

in-VM oracle: sanitizer / kernel fault / audited side effect

THE GATE

Hard accept

$C = 1 \wedge P = 1 \wedge D = 1 \rightarrow$ **accept** (within tested scope)

else → counterexample, revise again

not expressible → degrade honestly

the agent proposes a fixed invariant shape; the deterministic gate certifies it

From one case to reusable memory

The agent distills a solved case into a portable, evidence-linked reasoning pattern – stored as method, not answer – so the next investigation starts smarter.

1. EXPERIENCE

one solved case, compact

causal chain

reference → owner → dangling
created teardown use

root cause = missing scope check at reference creation

CVE-2022-2586 · cross-owner UAF · invariant set->table == table

- each claim bound to evidence: logs · source · observed release

2. DISTILL (hero)

abstract the mechanism, not the names

Recognize

signal: cross-owner reference + release relationship

Investigate

fixed chain: use → holder → binding → scope container

Judge

required constraint: referenced.scope == context.scope

Boundaries

stop if:
insufficient evidence
OR
another lifetime guarantee holds (refcount / RCU / shared ownership)

keep the causal relation · abstract the names (Owner A→Owner, nft_table→container)

3. ADMIT →|

a thin gate – three checks before memory accepts the entry

- ✓ explains THIS case
- ✓ rejects near-miss look-alikes
- ✓ transfers to a new case

record the tested scope – admit as scoped, not universal

4. MEMORY

the KB entry, now matched against future encounters

- **same mechanism** → **REUSE**
- **new evidence path** → **EXTEND**
- **distinct signal** → **NEW**

GOSIM

04

Does It Hold Up, and What's Next



Evaluation: 36 real Linux kernel cases

36 cases

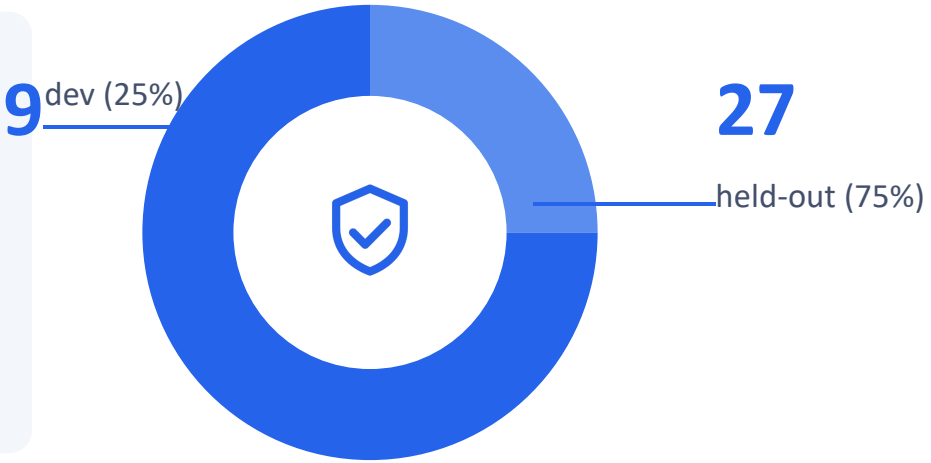
8 subtypes

9 / 27 dev / held-out

Benchmark split (by case count)

Vulnerability subtypes (8 total)

OOB	UAF	Race	Uninit
20	7	5	4



Official fixes are used only for post-hoc scoring

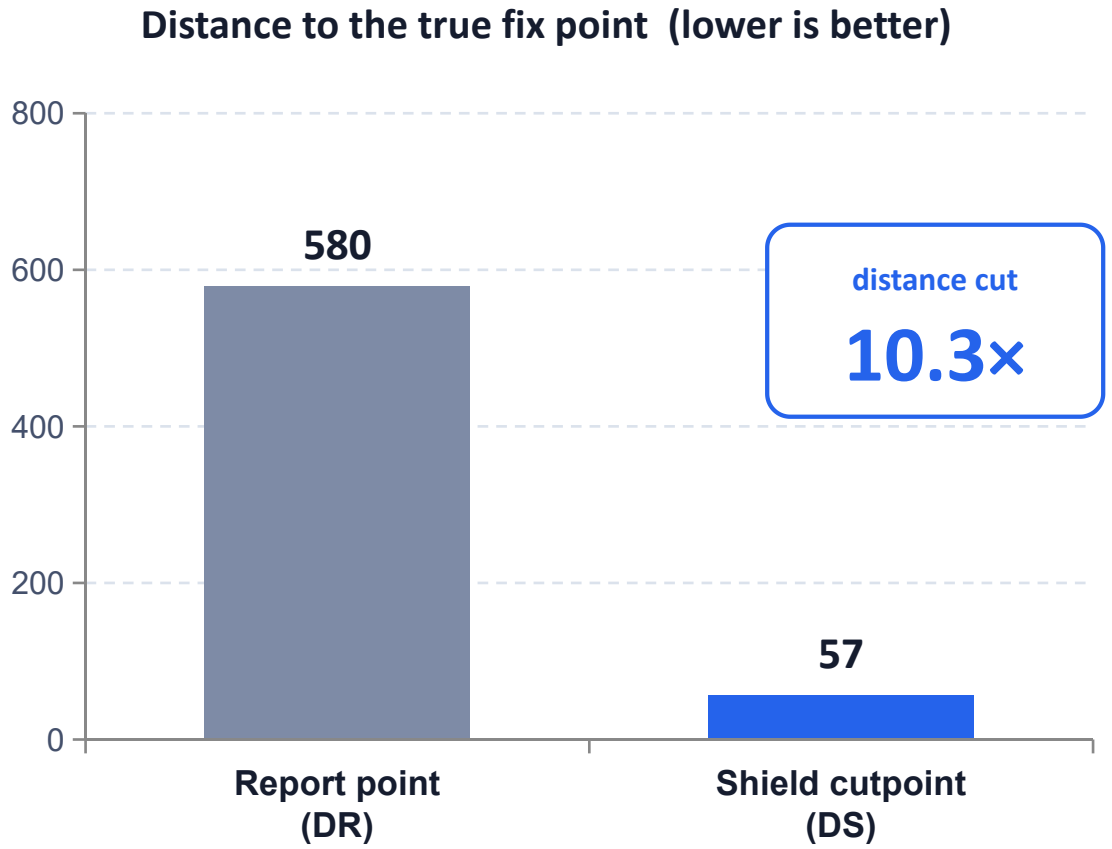
Results: closer to the root cause

36/36

semantically correct

- median DS 56.5 vs DR 579.5
- 32/36 cutpoints ≤ 0.5 × DR

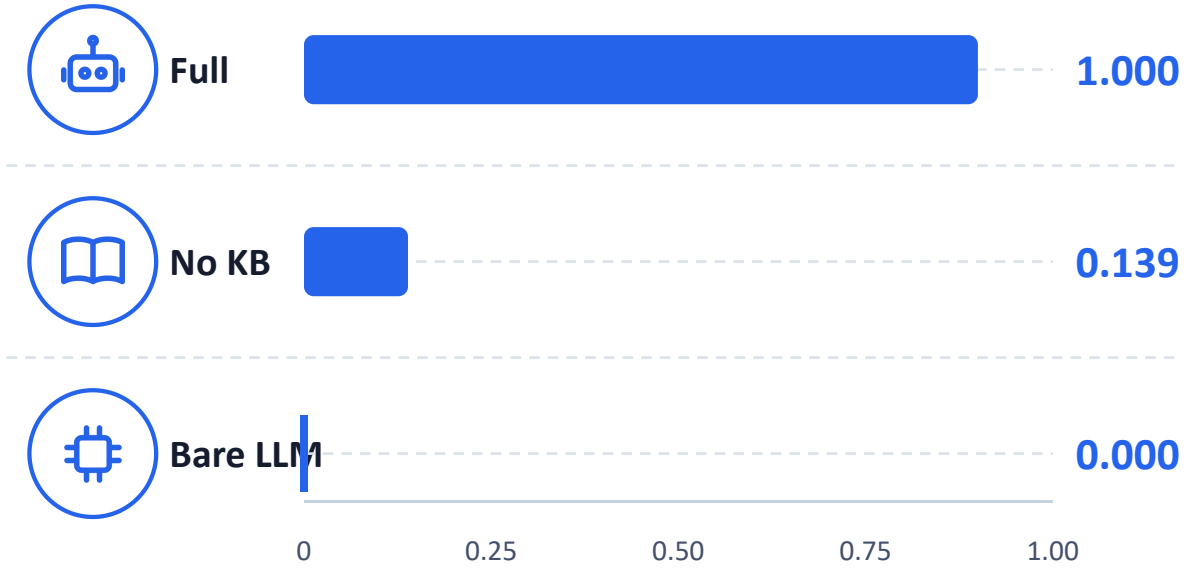
27 held-out cases also correct



DR: Report Point **DS:** Shield Cutpoint

Comparison & boundaries: structured agent beats bare prompting

Ablation accuracy



Known boundaries

- Boundary: variant test
- Only CVE-2022-2586
- No universal-defense guarantee

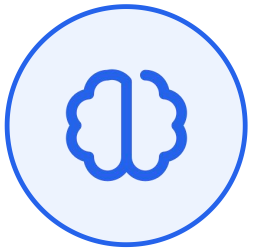
Runtime overhead

General workload	max 1.39%
Forwarding scenarios	2.41% – 7.42%

From “block this once” to “guard the root cause”

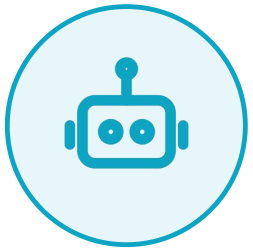
Build a reproducible emergency kernel-defense ecosystem — together with the GOSIM community

1 Root-cause shield synthesis



- Root-cause reasoning
- Selective, minimal shields

2 Agent + KB + runtime grounding



- KB-guided localization
- Runtime observability grounding

3 Adversarial self-test



- Adversarial self-test
- Publish only validated shields



Open Repository

- Validated shields only
- Auditable, reproducible, extensible

? Q&A